Informe III (Arquitectura del Computador)

Jesus Contreras (C.I: 31.021.750)

2026

1. Preguntas

1.1. Explique cómo se organiza la memoria cuando un sistema utiliza memory-mapped I/O. ¿En qué región de memoria se suelen mapear los dispositivos? ¿Qué implicaciones tiene para las instrucciones lw y sw?

En un sistema que utiliza memory-mapped I/O, el espacio de direcciones de memoria se comparte entre la memoria RAM y los registros de los dispositivos de entrada/salida. Esto significa que tanto la memoria como los dispositivos periféricos ocupan direcciones en un único espacio de direcciones.

Región de memoria típica: Los dispositivos suelen mapearse en las direcciones más altas del espacio de direcciones, reservando las direcciones bajas para la RAM. Por ejemplo, en MIPS32, las direcciones superiores a 0x7FFFFFFF en el espacio de usuario, o las direcciones en el segmento kseg1 (0xA0000000 - 0xBFFFFFFF) suelen utilizarse para dispositivos mapeados en memoria.

Implicaciones para lw y sw: Dado que los dispositivos se mapean en el mismo espacio de direcciones que la memoria, las instrucciones de carga (`lw`) y almacenamiento (`sw`) funcionan exactamente igual para acceder tanto a RAM como a dispositivos. Cuando se ejecuta un `lw` a una dirección que corresponde a un dispositivo, el hardware de bus redirige la solicitud al periférico correspondiente en lugar de a la memoria. Esto simplifica enormemente el juego de instrucciones, ya que no se necesitan instrucciones especializadas para E/S.

1.2. ¿Cuál es la principal diferencia entre memory-mapped I/O y la entrada/salida por puertos? ¿Qué ventajas y desventajas tiene cada enfoque? ¿Por qué MIPS32 utiliza principalmente memory-mapped I/O?

La principal diferencia radica en cómo se accede a los dispositivos:

- **Memory-mapped I/O:** Los dispositivos comparten el mismo espacio de direcciones que la memoria principal. Se accede a ellos mediante las mismas instrucciones de carga y almacenamiento que se usan para acceder a memoria.
- **E/S por puertos (Port-mapped I/O):** Los dispositivos tienen su propio espacio de direcciones separado, con instrucciones específicas para E/S (como `IN` y `OUT` en x86).

Ventajas y desventajas:

Característica	Memory-mapped I/O	E/S por puertos
Ventajas	- Instrucciones estándar de memoria - Gran espacio de direcciones - Protección mediante MMU - Instrucciones específicas claramente identificables	- No consume espacio de direcciones de memoria
Desventajas	- Consume espacio de direcciones - Puede requerir lógica adicional para distinguir accesos - Espacio de direcciones limitado	- Requiere instrucciones especiales

¿Por qué MIPS32 usa memory-mapped I/O? MIPS32 utiliza principalmente este enfoque porque simplifica el diseño del procesador al reducir

el número de instrucciones necesarias, manteniendo una arquitectura limpia y regular (arquitectura RISC). Además, al tener un espacio de direcciones de 32 bits, hay direcciones suficientes para mapear todos los periféricos necesarios.

1.3. En un sistema con memory-mapped I/O: ¿Qué problemas pueden surgir si dos dispositivos usan direcciones solapadas? ¿Cómo se evita este conflicto?

Si dos dispositivos usan direcciones solapadas (es decir, se asignan a la misma dirección física), pueden surgir conflictos graves:

- Cuando el procesador accede a esa dirección, múltiples dispositivos intentarían responder simultáneamente en el bus de datos, causando contención y potencialmente daños en los circuitos.
- Los datos serían corruptos, ya que cada dispositivo intentaría colocar su información en el bus al mismo tiempo.
- El comportamiento del sistema sería impredecible e inestable.

Cómo se evita este conflicto:

- Durante el diseño del sistema, se asigna un rango de direcciones único a cada dispositivo mediante decodificadores de direcciones.
- El decodificador de direcciones asegura que solo un dispositivo (o la memoria) sea seleccionado para cada dirección.
- Los mapas de memoria se definen cuidadosamente en la documentación del sistema y se implementan en hardware.
- En sistemas con buses expandibles (como PCIe), el firmware asigna dinámicamente direcciones durante el descubrimiento de dispositivos para evitar conflictos.

1.4. ¿Por qué se considera que el memory-mapped I/O simplifica el diseño del conjunto de instrucciones de un procesador? ¿Qué tipo de instrucciones adicionales serían necesarias si se usara E/S por puertos?

Memory-mapped I/O simplifica el diseño del conjunto de instrucciones porque:

- Elimina la necesidad de instrucciones especializadas de E/S, reduciendo la complejidad del decodificador de instrucciones.
- Mantiene la ortogonalidad del conjunto de instrucciones: todas las instrucciones de carga/almacenamiento funcionan igual independientemente del destino.
- Permite que los compiladores y programadores utilicen las mismas construcciones del lenguaje para acceder tanto a memoria como a dispositivos.

Instrucciones necesarias con E/S por puertos: Si se utilizara E/S por puertos, serían necesarias instrucciones adicionales como:

- `in` (leer de puerto)
- `out` (escribir a puerto)
- Posiblemente variantes para diferentes tamaños de datos (`inb`, `inw`, etc.)
- Instrucciones de sincronización específicas para puertos

Estas instrucciones incrementarían la complejidad del procesador y romperían la simplicidad del diseño RISC.

1.5. ¿Qué ocurre a nivel del bus de datos y direcciones cuando el procesador accede a una dirección de memoria que corresponde a un dispositivo? ¿Cómo sabe el hardware que debe acceder a un periférico en lugar de la RAM?

Cuando el procesador accede a una dirección que corresponde a un dispositivo:

1. La unidad de memoria genera la dirección en el bus de direcciones.
2. El decodificador de direcciones interpreta la dirección y determina que pertenece al rango asignado a un periférico específico.
3. La señal de selección de chip (chip select) correspondiente al dispositivo se activa, mientras que la señal de la RAM permanece inactiva.
4. El dispositivo (no la RAM) responde en el bus de datos:
   - Para una lectura, el dispositivo coloca sus datos en el bus.
   - Para una escritura, el dispositivo captura los datos del bus.
5. El procesador completa el ciclo de bus normalmente, sin saber si accedió a RAM o a un periférico.

El hardware sabe que debe acceder a un periférico en lugar de a la RAM mediante el decodificador de direcciones, que compara la dirección del bus con los rangos predefinidos y genera las señales de selección apropiadas.

1.6. ¿Es posible que un programa normal (sin privilegios) acceda a un dispositivo mapeado en memoria? ¿Qué mecanismos de protección existen para evitar accesos no autorizados?

¿Es posible? Sí, es posible técnicamente que un programa normal acceda a un dispositivo mapeado en memoria si no existen mecanismos de protección. Sin embargo, esto sería peligroso porque podría comprometer la estabilidad y seguridad del sistema.

Mecanismos de protección:

- **MMU con paginación:** La Unidad de Manejo de Memoria puede marcar las páginas que contienen direcciones de dispositivos como solo accesibles en modo kernel.
- **Anillos de protección:** Los procesadores implementan niveles de privilegio. En MIPS, los modos kernel y usuario determinan qué direcciones son accesibles.
- **Segmentación de memoria:** Algunas arquitecturas dividen la memoria en segmentos con diferentes privilegios (como kseg0, kseg1 en MIPS).
- **Sistema operativo:** El SO configura los mecanismos de hardware para que solo el kernel pueda acceder directamente a las direcciones de los dispositivos, proporcionando interfaces seguras (syscalls) para que los programas de usuario soliciten servicios de E/S.

1.7. ¿Qué técnicas se pueden emplear para evitar esperas activas innecesarias al interactuar con dispositivos?

Las esperas activas (polling) consumen CPU innecesariamente mientras se espera que un dispositivo complete una operación. Para evitarlas, se emplean:

- **Interrupciones:** El dispositivo notifica al procesador cuando está listo, permitiendo que la CPU ejecute otras tareas mientras tanto. El procesador suspende el programa actual, atiende al dispositivo y luego retoma la ejecución.
- **DMA (Acceso Directo a Memoria):** Un controlador DMA transfiere datos directamente entre el dispositivo y la memoria sin intervención de la CPU. La CPU solo configura la transferencia y recibe una interrupción al finalizar.
- **Buffering:** Se utilizan buffers para acumular datos, reduciendo la frecuencia de interacción con el dispositivo.
- **Programación asíncrona:** En lugar de esperar bloqueado, el programa registra callbacks o utiliza mecanismos de notificación que se activan cuando el dispositivo está listo.

- **Temporizadores:** Si se debe verificar periódicamente el estado del dispositivo, se utilizan temporizadores para espaciar las verificaciones, en lugar de hacer polling continuo.

2. Análisis y Discusión de los Resultados

Del análisis realizado sobre memory-mapped I/O, particularmente en el contexto de MIPS32, se pueden extraer las siguientes conclusiones:

- **Simplicidad vs. Complejidad:** El enfoque memory-mapped I/O demuestra cómo una decisión de diseño aparentemente simple (compartir el espacio de direcciones) puede simplificar drásticamente la arquitectura del procesador. MIPS32, como arquitectura RISC, se beneficia enormemente de esta simplificación al poder mantener un conjunto de instrucciones reducido y uniforme.

- **Compromisos de diseño:** Aunque memory-mapped I/O simplifica el procesador, traslada cierta complejidad al diseño del sistema (decodificadores de direcciones, gestión de conflictos) y al sistema operativo (protección, gestión de dispositivos). Este compromiso es aceptable porque permite que el hardware del procesador sea más eficiente y fácil de implementar.

- **Protección y seguridad:** La integración de dispositivos en el espacio de memoria hace crucial la implementación de mecanismos de protección robustos. Sin la MMU y los niveles de privilegio adecuados, cualquier programa podría interferir con dispositivos críticos, comprometiendo la estabilidad del sistema.

- **Eficiencia:** La combinación de memory-mapped I/O con técnicas avanzadas como interrupciones y DMA permite sistemas altamente eficientes donde la CPU puede dedicarse al procesamiento mientras las operaciones de E/S ocurren en paralelo.

- **Estandarización:** El enfoque memory-mapped I/O se ha convertido en el estándar de facto en procesadores modernos (ARM, MIPS, RISC-V) debido a su flexibilidad y a la abundancia de direcciones disponibles en sistemas de 32 y 64 bits.

En conclusión, memory-mapped I/O representa una solución elegante al problema de la comunicación con periféricos que, aunque tiene sus desafíos (como la gestión de conflictos de direcciones y la necesidad de protección), ofrece ventajas significativas en términos de simplicidad arquitectónica y flexibilidad de diseño.